

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 9. Regular Expression

Lecture: 97. Quantifiers

Section 1: Lecture Summary

This lecture introduces the concept of quantifiers in regular expressions, which are symbols used to specify how many times a character or group can repeat in a pattern. The main quantifiers covered are plus (+), asterisk (*), question mark (?), and curly braces {m} and {m,n}. The instructor explains the meaning of each quantifier, illustrating how they allow pattern matching for repeated characters or character sets with different repetition constraints. Examples are demonstrated using single characters, grouped strings (in parentheses), and character sets (in square brackets). The lecture also includes live Python code examples using the `re` module functions like `re.fullmatch()` and `re.findall()` to test these quantifiers on strings, emphasizing how matching works with repetitions and how the constraints affect valid matches. The key focus is understanding how quantifiers define pattern repetition limits in regex.

Section 2: Key Concepts and Explanations

- Quantifiers in Regular Expressions
 - Purpose: Define the allowed quantity of character or group repetitions in a pattern.
 - Common Quantifiers:
 - ``+`` (Plus): Matches one or more repetitions. Minimum 1, unlimited maximum.
 - Example: ``a+`` matches "a", "aa", "aaa", etc.
 - ``*`` (Asterisk): Matches zero or more repetitions. Can match an empty string.
 - Example: ``a`` matches "", "a", "aa", "aaa", etc.
 - ``?`` (Question Mark): Matches zero or one occurrence only.
 - Example: ``(ab)?`` matches "" or "ab".

- `{m}`: Matches exactly m repetitions.
 - Example: `[eimt]{4}` matches any 4-letter string from letters 'e', 'i', 'm', 't' (like "item", "emit").
- `{m,n}`: Matches at least m and at most n repetitions.
 - Example: `[eimt]{4,8}` matches length 4 to 8 strings from those letters.
 - Default for n if omitted: infinity (no upper bound).
- Grouping and Character Sets:
 - Parentheses `()` group multiple characters to apply quantifiers to the whole group.
 - Example: `(ab)+` means the string "ab" repeated one or more times.
 - Important: Repetitions apply to entire group, not individual characters.
 - Square brackets `[]` define a character set, matching any one character inside.
 - Example: `[abc]+` matches any string of a, b, c repeated one or more times.
- Summary of Quantifier Behavior

Quantifier	Meaning	Example Match
<code>+</code>	One or more	"a", "aa", "aaa"
<code>*</code>	Zero or more	"", "a", "aa", "aaa"
<code>?</code>	Zero or one	"", "ab"
<code>{m}</code>	Exactly m times	<code>[eimt]{4}</code> matches "time", "emit"
<code>{m,n}</code>	Between m and n times	<code>[eimt]{4,8}</code> matches strings 4 to 8

Section 3: Example Code and Use Cases

Example 1: full match with `(ab)?` - zero or one "ab"

Example 2: full match with `(ab)*` - zero or more "ab"

Example 3: find all occurrences of `[abc]+` - one or more a, b, c letters

Example 4: full match with `[eimt]{4}` - exactly 4 letters from set e, i, m, t

Example 5: full match with `[eimt]{4,8}` - 4 to 8 repetitions

```
import re
```

```

print(re.fullmatch('(ab)?', 'ab')) # Matches 'ab'
print(re.fullmatch('(ab)?', ''))   # Matches empty string

print(re.fullmatch('(ab)*', ''))    # Matches empty string
print(re.fullmatch('(ab)*', 'abababab')) # Matches repeated "ab"

text = '123 abc 987 bbcc@cbac'
matches = re.findall('[abc]+', text)
print(matches) # Output: ['abc', 'bbcc', 'cbac']

print(re.fullmatch('[eimt]{4}', 'item')) # Matches
print(re.fullmatch('[eimt]{4}', 'emit')) # Matches
print(re.fullmatch('[eimt]{4}', 'eeee')) # Matches
print(re.fullmatch('[eimt]{4}', 'iiii')) # Matches

print(re.fullmatch('[eimt]{4,8}', 'timeemit')) # Matches length 8 string
print(re.fullmatch('[eimt]{4,8}', 'itemti'))   # Matches length 7 string

```

These examples demonstrate matching patterns with various quantifiers, showing how they control repetition counts and what strings are considered valid matches.

Section 4: Key Takeaways

- Quantifiers specify how many times parts of a regex pattern can repeat.
- The plus '+' quantifier requires at least one occurrence, while the star '*' allows zero or more.
- The question mark '?' restricts repetition to zero or one occurrence only.
- Curly braces '{m}' enforce exact repetition count; '{m,n}' allows a range between minimum m and maximum n.
- Grouping '(' ')' applies quantifiers to whole sequences, not individual characters.
- Character sets '[' ']' match any one character from the given set.
- Quantifiers dramatically expand the kinds of strings a pattern can match, providing flexibility and power in regex design.
- Python's 're' module functions like 'fullmatch' and 'findall' let you test regex patterns comprehensively.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Regular Expressions

Quantifiers



Quantifiers

Quantifiers

Quantifiers

+

1 or more

Quantifiers

+ 1 or more

***** 0 or more

Quantifiers

+ 1 or more

***** 0 or more

? 0 or 1

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

default = 0

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

default = ∞

Quantifiers

+ 1 or more

***** 0 or more

? 0 or 1

{m} exactly m

{m,n} from m to n

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a'

'a'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a+'

'a'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a+'

'aa'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a+'

'aaa'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'a+'

'aaaaaaaaa'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'ab'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

'a'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

'ab'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

'abab'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

'ababababab'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(ab)+'

'ababababab'

'aaaabbbbb'

Quantifiers	pattern										
<table><tbody><tr><td>+</td><td>1 or more</td></tr><tr><td>*</td><td>0 or more</td></tr><tr><td>?</td><td>0 or 1</td></tr><tr><td>{m}</td><td>exactly m</td></tr><tr><td>{m,n}</td><td>from m to n</td></tr></tbody></table>	+	1 or more	*	0 or more	?	0 or 1	{m}	exactly m	{m,n}	from m to n	'(ab)+' 'ababababab' 'aaaabbbb'
+	1 or more										
*	0 or more										
?	0 or 1										
{m}	exactly m										
{m,n}	from m to n										

Quantifiers	pattern										
<table><tbody><tr><td>+</td><td>1 or more</td></tr><tr><td>*</td><td>0 or more</td></tr><tr><td>?</td><td>0 or 1</td></tr><tr><td>{m}</td><td>exactly m</td></tr><tr><td>{m,n}</td><td>from m to n</td></tr></tbody></table>	+	1 or more	*	0 or more	?	0 or 1	{m}	exactly m	{m,n}	from m to n	'[abc]'
+	1 or more										
*	0 or more										
?	0 or 1										
{m}	exactly m										
{m,n}	from m to n										

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]+'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]+'

'a'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]+'

'ab'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]+'

'abcaaaa'

Quantifiers

+ 1 or more

***** 0 or more

? 0 or 1

{m} exactly m

{m,n} from m to n

pattern

'[abc]+'

'abcaaaa'

'abcabcabcabcabc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

[abc]+

‘abcaaaa’

‘abcabcabcabcabc’

‘bbbccbaabbbccccc’

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

[abc]+

‘abcaaaa’

‘abcabcabcabcabc’

‘bbbccbaabbbccccc’

‘def ’

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]+'

'abcaaaa'

'abcabcabcabcabc'

'bbbccbaabbbccccc'

~~'def'~~

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]*'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]*'

' '

'c'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]*'

' '

'c'

'abcaaaa'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]*'

' '

'c'

'abcaaaa'

'abcabcabcabcabc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[abc]*'

' '

'c'

'abcaaaa'

'abcabcabcabcabc'

'bbbccbaabbbccccc'

Quantifiers

+ 1 or more

***** 0 or more

? 0 or 1

{m} exactly m

{m,n} from m to n

Quantifiers

+

1 or more

*

0 or more

?

0 or 1

{m}

exactly m

{m,n}

from m to n

pattern

Quantifiers

+

1 or more

*

0 or more

?

0 or 1

{m}

exactly m

{m,n}

from m to n

pattern

'abc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'abc'

'abc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(abc)'

'abc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(abc)?'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(abc)?'

' '

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'(abc)?'

' ' 'abc'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]'

Quantifiers

+ 1 or more

***** 0 or more

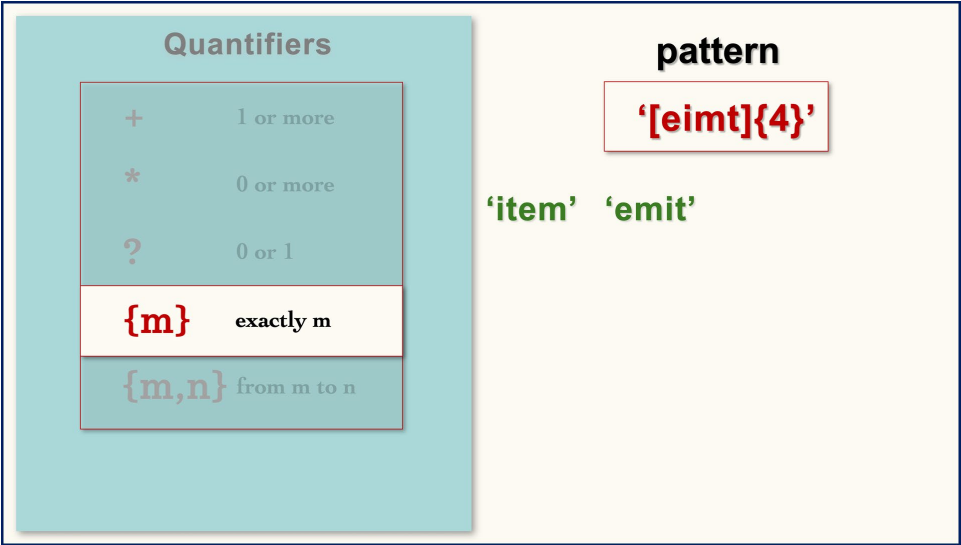
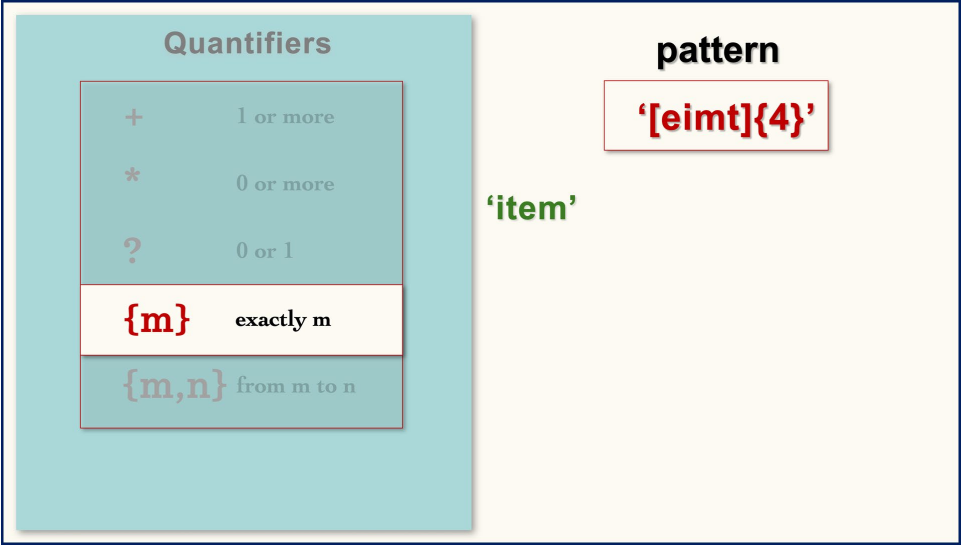
? 0 or 1

{m} exactly m

{m,n} from m to n

pattern

'[eimt]{4}'



Quantifiers	
+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4}'

'item' 'emit' 'mite'

Quantifiers	
+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4}'

'item' 'emit' 'mite' 'time'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4}'

'item' 'emit' 'mite' 'time'

'eimt'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4}'

'item' 'emit' 'mite' 'time'

'eimt' 'eeee'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4}'

'item' 'emit' 'mite' 'time'

'eimt' 'eeee' 'iiii'

Quantifiers

+	1 or more
*	0 or more
?	0 or 1
{m}	exactly m
{m,n}	from m to n

pattern

'[eimt]{4,}'

'item' 'emit' 'mite' 'time'

'eimt' 'eeee' 'iiii'

Quantifiers	pattern										
<table border="1"><tr><td>+</td><td>1 or more</td></tr><tr><td>*</td><td>0 or more</td></tr><tr><td>?</td><td>0 or 1</td></tr><tr><td>{m}</td><td>exactly m</td></tr><tr><td>{m,n}</td><td>from m to n</td></tr></table>	+	1 or more	*	0 or more	?	0 or 1	{m}	exactly m	{m,n}	from m to n	'[eimt]{4,}' 'item' 'emit' 'mite' 'time' 'eimt' 'eeee' 'iiii'
+	1 or more										
*	0 or more										
?	0 or 1										
{m}	exactly m										
{m,n}	from m to n										

Quantifiers	pattern										
<table border="1"><tr><td>+</td><td>1 or more</td></tr><tr><td>*</td><td>0 or more</td></tr><tr><td>?</td><td>0 or 1</td></tr><tr><td>{m}</td><td>exactly m</td></tr><tr><td>{m,n}</td><td>from m to n</td></tr></table>	+	1 or more	*	0 or more	?	0 or 1	{m}	exactly m	{m,n}	from m to n	'[eimt]{4,8}' 'item' 'emit' 'mite' 'time' 'eimt' 'eeee' 'iiii'
+	1 or more										
*	0 or more										
?	0 or 1										
{m}	exactly m										
{m,n}	from m to n										

